

ALPHA-RAG

ASP.NET Web API with RAG Endpoint

Retrieve · Embed · Reason · Respond

A complete record of prompts used and the AI outputs that shaped each stage of the .NET RAG service

Nandhini P · Nikitha S · Kalaiselvi V · Menmathi A

Department of Information Technology

VSB Engineering College

Prompt Engineering Documentation

Overview

This documentation presents the complete prompt-engineering trail used to design and build Alpha-RAG, an ASP.NET Core Minimal API that exposes a /chat endpoint performing Retrieval-Augmented Generation over a folder of Markdown documents. Alpha-RAG stands for Retrieve, Embed, Reason, and Respond — the four operating principles that govern every request the service handles.

The motivation for the project is simple: most production RAG examples in the wild are Python-only, which leaves enterprise .NET teams without a clear, idiomatic reference. Alpha-RAG demonstrates that the .NET ecosystem can deliver the same capabilities with first-class tooling, strong typing, dependency injection, and Swagger UI out of the box — while still supporting a fully local stack via ONNX embeddings and Ollama.

The objective of this document is to record, in order, each prompt issued to the AI assistant, summarise the AI output that resulted, and trace how those outputs were composed into the final service. The document is intended as a transparent reference for evaluation, reproducibility, and future extension.

How to read this document

- Each stage lists the Prompt Issued, an AI Output Summary, and the Objective met by that prompt.
- Prompts are numbered sequentially in the order they were used during construction.
- Outputs are summarised in plain language — full code listings live in the project repository.
- A traceability matrix at the end maps every prompt to the feature it produced and the Alpha-RAG principle it reinforces.

System at a glance

Alpha-RAG is a four-project .NET 8 solution. AlphaRag.Api hosts the Minimal API, dependency injection container, and Swagger UI. AlphaRag.Core owns the RAG pipeline (ingestion, chunking, retrieval, and the chat orchestrator). AlphaRag.Embeddings wraps the ONNX Runtime and the all-MiniLM-L6-v2 sentence-transformer model for fully local vectorisation. AlphaRag.Tests contains xUnit suites that run offline against a stubbed LLM. The corpus is any folder of Markdown files mounted at startup; reindexing is incremental and content-hashed.

Prompt 1 — Problem Framing and Architecture

Objective

Establish the scope, mission, and high-level architecture of the Alpha-RAG service before any code is written, and confirm that .NET is a viable host for an end-to-end RAG pipeline that matches the ergonomics of popular Python frameworks.

Prompt Issued

“Help me design an ASP.NET Core Minimal API called Alpha-RAG that exposes a /chat endpoint performing Retrieval-Augmented Generation over a folder of Markdown documents. It should use a local embedding model for vectorisation, support either OpenAI or a local Ollama LLM for generation, expose interactive Swagger UI for testing, and demonstrate that .NET is a first-class alternative to Python for RAG. Propose the architecture, NuGet packages, project layout, and a staged build plan.”

AI Output Summary

The AI returned a four-principle framing (Retrieve, Embed, Reason, Respond) and a stack built on .NET 8 Minimal APIs, Microsoft.SemanticKernel for orchestration, the ONNX Runtime with the all-MiniLM-L6-v2 embedding model for fully local vectorisation, and a pluggable IChatCompletionService that swaps between OpenAI and Ollama via configuration. It proposed a clean project layout — AlphaRag.Api for the web host, AlphaRag.Core for the RAG pipeline, AlphaRag.Embeddings for the ONNX wrapper, and docs/ as the seed Markdown corpus — and recommended Swashbuckle.AspNetCore for Swagger UI with example payloads. It outlined a five-stage build plan: scaffold the Minimal API, ingest and chunk the Markdown corpus, generate and persist embeddings in an in-memory vector store, wire the retrieval and LLM chain behind /chat, and finally expose Swagger with annotated request and response schemas. It also recommended an appsettings flag, UseLocalLlm, for deterministic offline tests.

Prompt 2 — Master System Prompt for Code Generation

Objective

Lock in the conventions the assistant must follow for every subsequent code-generation prompt so the project stays internally consistent across controllers, services, and tests.

Prompt Issued

“Write the master system prompt for the Alpha-RAG code generator. It must enforce a strict solution layout (AlphaRag.Api, AlphaRag.Core, AlphaRag.Embeddings, AlphaRag.Tests), require nullable reference types and implicit usings enabled in every csproj, mandate an IRagService interface with IngestAsync, RetrieveAsync, and ChatAsync methods, and define a ChatRequest / ChatResponse record pair used by the /chat endpoint. The prompt must forbid hard-coded API keys, require all secrets to flow through IConfiguration, and insist that every endpoint be documented with Swashbuckle XML comments and OpenAPI examples.”

AI Output Summary

The AI produced a system prompt that fixes the four-project solution layout, sets Nullable=enable and ImplicitUsings=enable as non-negotiable, and names the IRagService contract with its three async methods returning typed results (IngestReport, IReadOnlyList<RetrievedChunk>, ChatResponse). It defines ChatRequest(string Question, int? TopK) and ChatResponse(string Answer, IReadOnlyList<Citation> Citations, TimeSpan Elapsed) as immutable records, and lists hard rules: no secrets in source, all configuration via IConfiguration and User Secrets in development, every Minimal API endpoint decorated with .WithName, .WithOpenApi, and .Produces<T>, every service registered with the correct DI lifetime, and every unit test running offline with UseLocalLlm=true and a stubbed embedding generator. The prompt closes by requiring xUnit tests for the chunker, the cosine-similarity retriever, and the end-to-end /chat happy path before any feature is considered complete.

Prompt 3 — Stage 1 — Minimal API Scaffold

Objective

Bootstrap the .NET 8 solution with a runnable Minimal API, Swagger UI, dependency injection wiring, and a health endpoint so later stages can plug in features without re-deriving infrastructure.

Prompt Issued

“Generate the Stage 1 scaffold for Alpha-RAG: a .NET 8 Minimal API project with Swashbuckle wired up, a /health endpoint returning service version and uptime, an appsettings.json with sections for Embeddings, Llm, and Corpus, and a Program.cs that registers placeholder services for IRagService, IEmbeddingService, and IVectorStore. Include a Directory.Build.props enabling Nullable and ImplicitUsings across the solution.”

AI Output Summary

The AI delivered a runnable solution: Program.cs uses the WebApplication builder with AddEndpointsApiExplorer, AddSwaggerGen with XML comments included, and a fluent registration block for the three placeholder services. The /health endpoint returns a typed HealthResponse record carrying version, environment, and uptime computed from a captured Stopwatch. appsettings.json exposes EmbeddingsOptions (ModelPath, Dimension), LlmOptions (Provider, Model, Endpoint, ApiKeySecret), and CorpusOptions (DocsPath, ChunkSize, ChunkOverlap), each bound via Options pattern. Directory.Build.props centralises LangVersion=latest, Nullable=enable, ImplicitUsings=enable, and TreatWarningsAsErrors=true so every project inherits the same strictness.

Prompt 4 — Stage 2 — Markdown Ingestion and Chunking

Objective

Convert a folder of Markdown documents into clean, overlap-aware text chunks that are ready to be embedded, preserving section headings as metadata for citation.

Prompt Issued

“Implement a MarkdownIngestionService that recursively scans CorpusOptions.DocsPath, parses each .md file with Markdig, strips front-matter and code-fence noise, and produces a stream of Chunk records (Id, SourcePath, Heading, Order, Text, ContentHash). Use a token-aware sliding window with ChunkSize and ChunkOverlap from configuration, and skip files whose hash has not changed since the last run.”

AI Output Summary

The AI generated MarkdownIngestionService implementing IIngestionService with an async IEnumerable<Chunk> pipeline. Markdig is used with the advanced extensions pipeline; a custom renderer walks the AST so headings become Chunk.Heading and code fences are preserved verbatim but trimmed of trailing whitespace. A simple whitespace tokenizer drives the sliding window, and ChunkOverlap is clamped to half of ChunkSize to prevent runaway duplication. A small JSON manifest at .alpha-rag/ingest-manifest.json stores per-file SHA-256 hashes, enabling incremental reindex. Unit tests cover empty files, files with only front-matter, very long single-paragraph documents, and boundary conditions on overlap.

Prompt 5 — Stage 3 — Local ONNX Embedding Service

Objective

Vectorise every chunk with a fully local, deterministic embedding model so the service can run offline and produce reproducible test results.

Prompt Issued

“Build an `OnnxEmbeddingService` that loads the `all-MiniLM-L6-v2` ONNX model via `Microsoft.ML.OnnxRuntime`, tokenises input with the `BertTokenizer` NuGet package, runs inference in batches, mean-pools the token embeddings, L2-normalises the result, and returns `float[]` vectors of length 384. Expose it behind an `IEmbeddingService` interface, register it as a singleton, and add a `StubEmbeddingService` for tests that returns a hash-derived vector.”

AI Output Summary

The AI produced `OnnxEmbeddingService` with a constructor that loads the `InferenceSession` once and reuses it, a `BatchEmbedAsync` method that processes inputs in configurable batch sizes (default 16) to bound memory, and a mean-pooling routine that respects the attention mask so padding tokens do not bias the vector. L2 normalisation is performed in-place. `StubEmbeddingService` implements the same contract by deriving 384 floats from a SHA-256 of the input — deterministic, fast, and offline-safe. DI registration uses a factory that selects between `Onnx` and `Stub` based on `EmbeddingsOptions.Provider`, so xUnit tests opt into the stub via a single configuration override.

Prompt 6 — Stage 4 — Vector Store and Cosine-Similarity Retriever

Objective

Provide a fast, dependency-free retriever that ranks chunks by cosine similarity and returns the top-K matches with their source metadata for citation.

Prompt Issued

“Create an `InMemoryVectorStore` implementing `IVectorStore` (`UpsertAsync`, `SearchAsync`, `CountAsync`) backed by a thread-safe list of `(Chunk, float[])` records. Implement `SearchAsync` as a cosine-similarity scan that returns `IReadOnlyList<RetrievedChunk>` ordered by score, with a configurable `TopK` (default 5). Add a snapshot-to-disk option that serialises the store to a single `.bin` file for warm starts, and benchmark it against a 10,000-chunk corpus.”

AI Output Summary

The AI delivered `InMemoryVectorStore` using a `ReaderWriterLockSlim` to allow concurrent reads during search and exclusive writes during ingest. `SearchAsync` uses a single pass that computes cosine similarity (vectors are already L2-normalised, so this reduces to a dot product) and maintains a fixed-size min-heap of TopK results, avoiding a full sort. The snapshot format is a versioned binary layout: header, chunk metadata table, then a contiguous float matrix for SIMD-friendly loads. A `BenchmarkDotNet` harness reports sub-15 ms p95 search latency over 10,000 chunks on a laptop CPU, with cold-start load under 250 ms from the snapshot.

Prompt 7 — Stage 5 — LLM Chain and /chat Endpoint

Objective

Wire the retrieved context into a grounded prompt, call the configured LLM (OpenAI or Ollama), and return an answer with citations behind the public /chat endpoint.

Prompt Issued

“Implement RagService.ChatAsync: embed the question, retrieve TopK chunks, build a grounded system prompt that instructs the model to answer only from the provided context and to cite chunks by [n] markers, call IChatCompletionService (configured for either OpenAI gpt-4o-mini or Ollama llama3), and return a ChatResponse with Answer, Citations, and Elapsed. Expose it as POST /chat in Program.cs with full OpenAPI annotations.”

AI Output Summary

The AI generated RagService that composes the pipeline in a single ChatAsync method: a Stopwatch is started, the question is embedded, the top chunks are retrieved, a system prompt is rendered from a Scriban template (Templates/grounded-chat.sbn) that enumerates chunks as [1], [2], [3] with source path and heading, the chat history is sent to IChatCompletionService, and the response is parsed to extract citation markers which are then matched back to the retrieved chunks. The /chat endpoint is registered with .WithName("Chat"), .WithOpenApi(), .Accepts<ChatRequest>("application/json"), and .Produces<ChatResponse>(StatusCodes.Status200OK). OpenAI and Ollama implementations of IChatCompletionService share a thin HttpClient base, with retry and timeout configured via Polly.

Prompt 8 — Stage 6 — Swagger UI, Examples, and xUnit Tests

Objective

Make the API self-documenting and verifiable: rich Swagger UI with example payloads, plus an xUnit suite that proves every layer works offline.

Prompt Issued

“Enrich Swagger with example ChatRequest and ChatResponse payloads using Swashbuckle.AspNetCore.Filters, add XML doc comments to every public type, and expose Swagger UI at the application root in Development. Then write xUnit tests for the chunker (boundary cases), the cosine-similarity retriever (ordering and TopK), the OnnxEmbeddingService contract via StubEmbeddingService, and a WebApplicationFactory-based integration test that POSTs to /chat with UseLocalLlm=true and asserts on the citation count.”

AI Output Summary

The AI added IExamplesProvider implementations for ChatRequest and ChatResponse, wired AddSwaggerExamplesFromAssemblyOf<Program>(), and configured SwaggerUI with RoutePrefix=string.Empty so the docs land at the root URL in Development. Every public record and interface received triple-slash XML comments and the csproj enabled GenerateDocumentationFile. The xUnit project uses Microsoft.AspNetCore.Mvc.Testing to spin up the API in-process with an overridden configuration that selects StubEmbeddingService and a deterministic FakeChatCompletionService; the integration test asserts a 200 response, non-empty Answer, and that every [n] marker resolves to a citation. CI runs the entire suite in under eight seconds with zero network access.

Traceability Matrix

The matrix below maps each prompt to the feature it produced and the Alpha-RAG principle it reinforces.

Prompt #	Stage / Feature	Alpha-RAG Principle
1	Problem framing & architecture	Retrieve
2	Master system prompt	Retrieve · Respond
3	Minimal API scaffold	Respond
4	Markdown ingestion & chunking	Retrieve
5	Local ONNX embedding service	Embed
6	Cosine-similarity retriever & vector store	Retrieve
7	LLM chain (OpenAI / Ollama) & /chat endpoint	Reason · Respond
8	Swagger UI, examples & xUnit tests	Respond

Closing Notes

Every prompt above was issued to a single AI assistant working under the master system prompt defined in Prompt 2. Outputs were reviewed, lightly edited for project conventions, and committed alongside the tests they enabled. The result is a .NET-native RAG service whose behaviour can be retraced — from a Markdown document, through chunking and embedding, into the vector retrieval step, through the LLM generation call, and out to the Swagger-documented /chat response — back to the original prompt that shaped each layer.

Alpha-RAG stands as evidence that ASP.NET Core is a complete, production-ready platform for building Retrieval-Augmented Generation services. With Swagger UI, dependency injection, strong typing, and configurable local or cloud LLMs available from day one, .NET teams no longer need to leave their stack to deliver modern, grounded AI experiences.